T JOHN INSTITUTE OF TECHNOLOGY

HACK2HIRE 1.0

NetSense AI

Automated WiFi Experience Analyzer

Team Name: byte-warriors

Team Leader: Abhishek
Team Member: Savan M K

Date: April 23, 2026
Status: Final Submission

Project Brief & Documentation
Submitted for the Hack2Hire 1.0 Build Phase
(April 16 - April 23, 2026)

1 Executive Summary

NetSense AI is an AI-powered network performance analysis web application designed to help users understand the quality of their internet connection in a simple, visual, and intelligent way. Moving beyond traditional speed tests that only display raw telemetry data (Ping, Download, Upload), NetSense AI contextualizes network metrics into practical use-case readiness.
Featuring a modern, dark-themed, glassmorphic UI, real-time diagnostic animations, and an integrated Google Gemini AI chatbot, the platform analyzes current network health and clearly explains whether the connection is suitable for streaming, gaming, video calls, and large cloud uploads.

2 Problem Statement

In an era where digital connectivity is the backbone of daily operations, the average consumer struggles to interpret technical network diagnostics.

- **DataDisconnect:** Standardnetworkmonitorsoutputrawnumbers(e.g.,“45Mbps”,“70ms ping”) which hold no semantic meaning for non-technical users.
- **Lack of Context:** A user experiencing lag during a Zoom call or buffering on Netflix does not know if their 15 Mbps upload speed is the root cause.
- **Absence of Guidance:** When a network issue occurs, users are left without actionable, easy-to-understand steps to optimize or troubleshoot their connection.

3 The Innovation: Proposed Solution

To solve this gap, **byte-warriors** developed **NetSense AI**. The innovation lies in bridging raw metrics with human-readable intelligence.

1. **Contextual Network Readiness:** Instead of just displaying “30 Mbps”, NetSense translates this to: *“Ready for HD Streaming, but weak for competitive gaming.”*

2. **Algorithmic Health Scoring:** The system calculates a dynamic Network Health Score (0-100) based on weighted thresholds for latency and throughput, assigning visual grades (Excellent, Good, Fair, Poor).

3. **AI-Powered Diagnostics:** An integrated Google Gemini chatbot reads the live network metrics and provides conversational troubleshooting. Users can ask natural questions like, *“Why is my ping so high?”* or *“Can I present a video meeting right now?”* and receive hyper-personalized advice based on their current connection state.

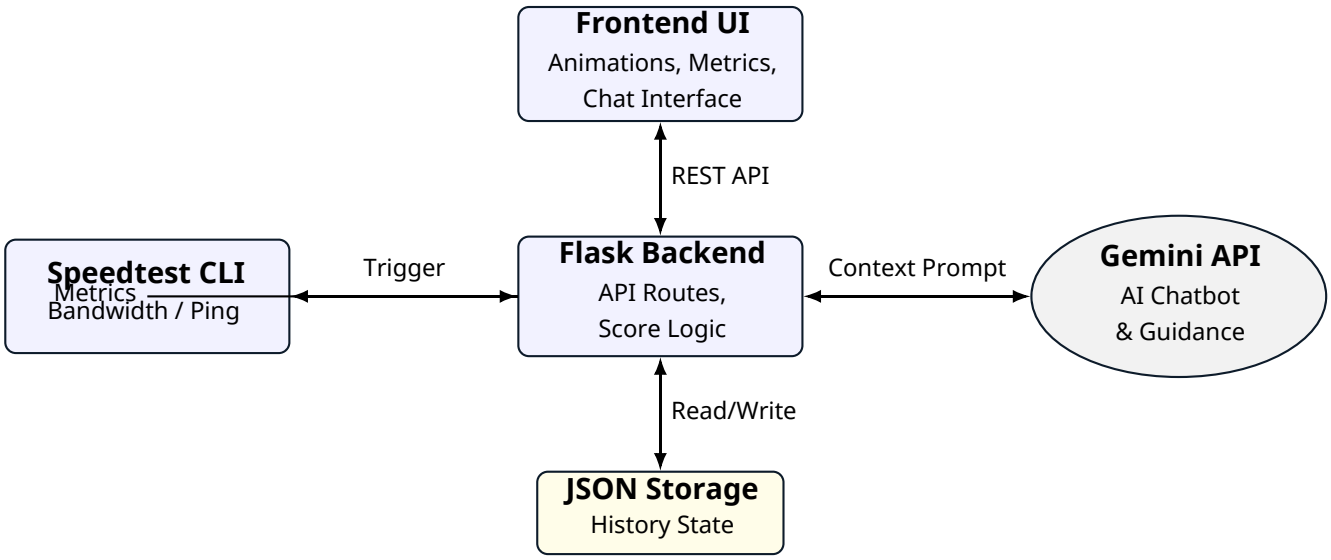
4 Technology Stack

Our application is designed as a robust, full-stack Flask web project, separated cleanly into a frontend UI layer and a backend processing engine.

- **Frontend:** HTML5, CSS3 (Custom Animations & Glassmorphism), Vanilla JavaScript, Jinja2 Templating.
- **Backend:** Python 3, Flask, Gunicorn (Production WSGI).
- **Network Core:** speedtest-cli for real-world telemetry harvesting.
- **AI & Logic Engine:** Google Gemini API (gemini-2.5-flash), Custom Health Scoring Algorithms.
- **DataPersistence:** JSONfilestorage(data.json,history.json)forrecentdiagnosticcaching.
- **Deployment:** Render (Free Tier), GitHub Integration.

5 System Architecture Flow

The architecture is carefully designed to handle asynchronous operations efficiently, ensuring a smooth and responsive user experience at all times. Instead of blocking the interface, resource-intensive tasks such as network speed testing and AI prediction processing are executed in the background. This allows the frontend to remain interactive, responsive, and visually stable while computations are being performed. The system uses a non-blocking workflow where user actions trigger backend processes without freezing the UI. Meanwhile, progress animations and status indicators keep the user informed during processing.
Additionally, the backend manages these tasks using controlled intervals and optimized execution to avoid performance bottlenecks, especially in cloud environments. Data is continuously processed and updated without requiring manual refreshes, enabling real-time insights. The architecture also ensures that heavy operations like speed testing do not interfere with user interactions or degrade overall performance. This separation of concerns between frontend responsiveness and backend processing enhances scalability, reliability, and user experience. Overall, the system delivers a seamless interaction flow while efficiently handling complex operations behind the scenes.



6 Core Features

- **Interactive Diagnostic Radar:** A custom CSS-animated scanning interface that visually represents the network phase (Latency check, Download check, Upload check).
- **Smart Readiness Cards:** Dynamic status indicators detailing the connection's suitability for Streaming, Gaming, Video Calls, and Cloud Uploads.
- **AI-PoweredNetworkAssistant:** AconversationalinterfacedrivenbyGoogleGemini. The chatbot is context-aware, meaning its prompt is injected with the user's latest network statistics prior to answering.
- **Simulated "Fix Network" Flow:** An automated troubleshooting guide that generates safe, actionable advice (e.g., DNS changing, router placement) based on the detected net- work bottlenecks.
- **Historical Snapshots:** Caches the latest 10 scans to provide users a timeline view of their connection stability.

7 DiagnosticWorkflow

The user experience is built to be completely frictionless. The end-to-end flow executes as follows:

1. The user clicks **"Test Performance"** on the dashboard.
2. The frontend triggers the radar animation and sends a POST /trigger-performance request to the backend.
3. The Flask backend delegates the task to network_monitor.py, invoking speedtest-cli.
4. Uponreturningmetrics,thebackendcomputesthe*HealthScore(0-100)*anddetectsthresh-old breaches (e.g., Ping ≥ 70ms).
5. The backend caches the result in data.json and history.json.
6. The frontend receives the JSON payload, gracefully animating the metric cards, updating the readiness badges, and populating the Quick Guidance panel with AI text.

8 Setup & Deployment Instructions

NetSense AI is designed to be easily deployable in cloud environments.

Local Setup:

1. Clone repository: git clone <Repo-URL>
2. Create Virtual Env: python -m venv venv && source venv/bin/activate
3. Install dependencies: pip install -r requirements.txt
4. Add environment variables to .env: GEMINI_API_KEY=your_api_key_here
5. Run the app: python backend/app.py

Production Deployment (Render): The application is deployed via Render using a Web Service configuration. A Procfile and render.yaml are configured to utilize Gunicorn with ex- tended timeouts (--timeout 180) to accommodate the physical time required to execute speed tests.

9 Limitations & Future Scope

- **Client-Side Shift:** Currently, hosting the app on Render means the speedtest analyzes the server's network, not the end-user's local WiFi. Future iterations will migrate the testing protocol entirely to the client-side browser using WebRTC or WebSockets.
- **PersistentStorage:** MigratingfromJSONfilestoragetoapersistentPostgreSQLdatabase for user accounts and long-term network history tracking.
- **Advanced Diagnostics:** Implementing packet loss and jitter estimation for a more robust gaming readiness score.